

WanderLust Backend Project

A Node.js + Express + MongoDB backend project for creating and managing **Listings**.

This project uses:

- Node.js
- npm
- Express.js
- MongoDB
- Mongoose
- EJS
- EJS-Mate
- Method-Override
- Joi
- Express-Session
- Connect-Flash
- Cookie-Parser

The project covers RESTful routing, CRUD operations, server-side validation, sessions, cookies, flash messages, middleware, and EJS layouts.

■ Project Setup and Package Installation

1. Initialize the Node.js Project

Create `package.json`:

```
npm init -y
```

2. Install the Main Direct Dependencies

You can install all packages used directly by the application with one command:

```
npm i express mongoose ejs ejs-mate method-override joi express-session connect-flash cookie-parser
```

Or install them individually:

```
npm i express
npm i mongoose
npm i ejs
npm i ejs-mate
npm i method-override
npm i joi
npm i express-session
npm i connect-flash
```

```
npm i cookie-parser
```

3. Check Directly Installed Packages

```
npm list --depth=0
```

This shows the packages directly installed in the current project.

■ Node.js and npm

What is Node.js?

Node.js is a JavaScript runtime that allows JavaScript to run outside the browser.

Example:

```
node app.js
```

What is npm?

npm stands for **Node Package Manager**.

It is used to install and manage third-party packages.

Example:

```
npm i express
```

npm manages files such as:

```
package.json  
package-lock.json  
node_modules/
```

■ Direct Packages vs Dependency Packages

There is an important difference between packages that your application directly uses and packages installed automatically as dependencies.

Direct Dependencies

These are the packages your application intentionally installs and uses:

```
express  
mongoose
```

```
ejs
ejs-mate
method-override
joi
express-session
connect-flash
cookie-parser
```

Transitive Dependencies

When you install a package, npm may automatically install other packages required by that package.

Examples that may appear in `node\_modules` include:

```
uid-safe
safe-buffer
randombytes
on-headers
```

These are generally dependencies of other packages.

You normally **do not need to install them manually** unless your own application directly imports and uses them.

The exact dependency tree can change between package versions.

A simple dependency relationship can look like:

```
Your Application
|
+-- express-session
|   |
|   +-- uid-safe
|       |
|       +-- randombytes
|
+-- other packages
    |
    +-- their dependencies
```

Therefore, having 100+ folders inside `node\_modules` does **not** mean you personally installed 100+ packages.

■ Node.js Built-in Modules

Node.js also provides built-in modules.

These are included with Node.js and do not require `npm install`.

Common examples:

```
const path = require("path");
const fs = require("fs");
const crypto = require("crypto");
const http = require("http");
const os = require("os");
const url = require("url");
```

Do **not** install these with npm just because you see them imported.

For example:

```
npm i path
```

is normally unnecessary because `path` is already part of Node.js.

■ Express.js

Express is the backend web framework used to create the server, routes, and middleware.

Installation

```
npm i express
```

Basic Setup

```
const express = require("express");

const app = express();

app.listen(8080, () => {
  console.log("Server started on port 8080");
});
```

Express allows us to create routes such as:

```
app.get("/", (req, res) => {
  res.send("Hello World!");
});
```

■■ MongoDB + Mongoose

MongoDB

MongoDB is the database used by the WanderLust application.

Mongoose

Mongoose is an ODM (Object Data Modeling) library for MongoDB and Node.js.

It allows us to define schemas and models and work with MongoDB using JavaScript objects.

Installation

```
npm i mongoose
```

Connection

Example:

```
const mongoose = require("mongoose");

mongoose
  .connect("mongodb://127.0.0.1:27017/wanderlust")
  .then(() => {
    console.log("Connected to MongoDB");
  })
  .catch((err) => {
    console.log(err);
  });
```

Here:

```
127.0.0.1  -> Local computer
27017      -> Default MongoDB port
wanderlust -> Database name
```

■ Listing Model

A Listing represents a property or place stored in MongoDB.

A listing can contain:

- Title
- Description
- Image
- Location
- Country

Example:

```
const listingSchema = new mongoose.Schema({
  title: String,
  description: String,
  image: String,
  location: String,
  country: String,
});
```

A Mongoose model can then be created from the schema:

```
const Listing = mongoose.model("Listing", listingSchema);

module.exports = Listing;
```

■ CRUD Operations

CRUD means:

	Meaning	HTTP Method
	Create a listing	POST
	View listing(s)	GET
	Update a listing	PUT
	Delete a listing	DELETE

Typical RESTful routes:

```
GET    /listings
GET    /listings/new
POST   /listings
GET    /listings/:id
GET    /listings/:id/edit
PUT    /listings/:id
DELETE /listings/:id
```

■■ RESTful Routes

RESTful routing organizes application URLs around resources.

For the WanderLust project, the main resource is:

```
/listings
```

Examples:

```
GET    /listings
```

Shows all listings.

```
GET    /listings/new
```

Shows the form for creating a listing.

```
POST   /listings
```

Creates a new listing.

```
GET    /listings/:id
```

Shows one listing.

```
GET    /listings/:id/edit
```

Shows the edit form.

```
PUT    /listings/:id
```

Updates a listing.

```
DELETE /listings/:id
```

Deletes a listing.

■ Method-Override

HTML forms normally support:

```
GET
POST
```

But RESTful applications also need:

```
PUT
DELETE
```

The `method-override` package allows us to use these methods from HTML forms.

Installation

```
npm i method-override
```

Require the Package

In `app.js`:

```
const methodOverride = require("method-override");
```

Add the middleware:

```
app.use(methodOverride("_method"));
```

Usage in `edit.ejs`

```
<form method="POST" action="/listings/<%= listing._id %>?_method=PUT">
```

The browser sends a POST request, and `method-override` changes it to:

```
PUT
```

This allows the application to update the listing.

For deletion:

```
<form method="POST" action="/listings/<%= listing._id %>?_method=DELETE">
```

The request is treated as:

```
DELETE
```

■ EJS

EJS stands for **Embedded JavaScript Templates**.

It allows JavaScript data to be inserted into HTML.

Installation

```
npm i ejs
```

Setup

```
app.set("view engine", "ejs");
```

Example:

```
<h1><%= listing.title %></h1>
```

If the title is:

```
Beautiful House
```

the browser displays:

```
Beautiful House
```

■ EJS-Mate

What is EJS-Mate?

EJS-Mate is a layout system for EJS.

It allows us to create a common layout instead of repeating the same HTML on every page.

A common layout can contain:

- Navbar

- Footer
- ``
- CSS links
- Bootstrap links
- Common page structure

Installation

```
npm i ejs-mate
```

Setup in `app.js`

```
const engine = require("ejs-mate");  
  
app.engine("ejs", engine);  
app.set("view engine", "ejs");
```

Example layout usage:

```
<% layout("/layouts/boilerplate") -%>
```

■ Express-Session

Express-Session is used to create and manage sessions.

Sessions are useful for storing temporary information between requests, such as authentication/session data.

Installation

```
npm i express-session
```

Require

```
const session = require("express-session");
```

Middleware

```
app.use(  
  session({  
    secret: "mysupersecret",  
    resave: false,  
    saveUninitialized: true,  
  })  
);
```

For a real production application, use a strong secret and configure the session store appropriately rather than relying on the default in-memory store.

■ Connect-Flash

What is Connect-Flash?

Flash messages are temporary messages that are commonly displayed after a redirect.

Examples:

```
Listing created successfully!  
Listing updated successfully!  
Listing deleted successfully!
```

A flash message is generally available for the next request and then removed.

Installation

```
npm i connect-flash
```

Require

```
const flash = require("connect-flash");
```

Flash messages require a session.

Example:

```
app.use(  
  session({  
    secret: "mysupersecret",  
    resave: false,  
    saveUninitialized: true,  
  })  
);  
  
app.use(flash());
```

Set a Flash Message

```
req.flash("success", "Listing created successfully!");
```

Read a Flash Message

```
req.flash("success");
```

Example:

```
app.get("/flash", function (req, res) {  
  req.flash("info", "Flash is back!");  
  res.redirect("/");  
});
```

```
});  
app.get("/", function (req, res) {  
  res.render("index", {  
    messages: req.flash("info"),  
  });  
});
```

■ Cookie-Parser

What is Cookie-Parser?

`cookie-parser` is Express middleware used to parse cookies from incoming HTTP requests.

Installation

```
npm i cookie-parser
```

Require

```
const cookieParser = require("cookie-parser");
```

Middleware

```
app.use(cookieParser());
```

Cookies can then be accessed through:

```
req.cookies
```

Example:

```
console.log(req.cookies);
```

■ Joi Schema Validation

Joi is used for server-side validation.

It checks incoming data before that data is saved to the database.

Installation

```
npm i joi
```

Example Without Joi

Without a validation library, we might write:

```
if (!newListing.title) {
  throw new ExpressError(400, "Title is missing!");
}

if (!newListing.description) {
  throw new ExpressError(400, "Description is missing!");
}

if (!newListing.location) {
  throw new ExpressError(400, "Location is missing!");
}
```

This can become difficult to maintain when there are many fields.

Joi Schema

```
const Joi = require("joi");

const listingSchema = Joi.object({
  title: Joi.string().required(),
  description: Joi.string().required(),
  location: Joi.string().required(),
  country: Joi.string().required(),
  image: Joi.string().allow("", null),
});
```

Validate Data

```
const { error } = listingSchema.validate(req.body);

if (error) {
  throw new ExpressError(400, error.details[0].message);
}
```

Why Use Joi?

Joi helps us:

- Validate user input
- Make required fields mandatory
- Prevent invalid data from reaching the database
- Keep validation organized
- Generate useful validation messages

■■ Express Error Handling

A custom error class can be used to create consistent application errors.

Example:

```
class ExpressError extends Error {
  constructor(statusCode, message) {
    super();
    this.statusCode = statusCode;
    this.message = message;
  }
}

module.exports = ExpressError;
```

Example usage:

```
throw new ExpressError(404, "Listing not found!");
```

An Express error-handling middleware can then handle the error.

Example:

```
app.use((err, req, res, next) => {
  const { statusCode = 500, message = "Something went wrong!" } = err;

  res.status(statusCode).send(message);
});
```

■ Middleware

Middleware functions run during the request-response cycle.

Common middleware in this project includes:

```
app.use(express.urlencoded({ extended: true }));
app.use(methodOverride("_method"));
app.use(cookieParser());
app.use(session(...));
app.use(flash());
```

A middleware function generally has:

```
(req, res, next)
```

Example:

```
app.use((req, res, next) => {
  console.log("Middleware running");
  next();
});
```

`next()` passes control to the next middleware or route.

■ Main Packages Used

Package	Purpose
`express`	Backend web framework
`mongoose`	MongoDB ODM
`ejs`	Template engine
`ejs-mate`	EJS layouts
`method-override`	PUT/DELETE requests from HTML forms
`joi`	Server-side validation
`express-session`	Session management
`connect-flash`	Temporary flash messages
`cookie-parser`	Cookie parsing

■ Basic Project Structure

```
project/
■
■ ■ ■ models/
■   ■ ■ ■ listing.js
■
■ ■ ■ views/
■   ■ ■ ■ layouts/
■     ■ ■ ■ boilerplate.ejs
■     ■
■   ■ ■ ■ listings/
■     ■ ■ ■ index.ejs
■     ■ ■ ■ new.ejs
■     ■ ■ ■ edit.ejs
■     ■ ■ ■ show.ejs
■
■ ■ ■ public/
■   ■ ■ ■ css/
■   ■ ■ ■ js/
■
■ ■ ■ utils/
■   ■ ■ ■ ExpressError.js
■
■ ■ ■ app.js
■ ■ ■ package.json
■ ■ ■ package-lock.json
```

■■ Technologies Used

Technology	Purpose
Node.js	JavaScript runtime
npm	Package manager
Express.js	Backend framework
MongoDB	Database
Mongoose	MongoDB ODM

Technology	Purpose
EJS	Template engine
EJS-Mate	EJS layouts
Method-Override	PUT/DELETE requests from HTML forms
Joi	Data validation
Express-Session	Session management
Connect-Flash	Flash messages
Cookie-Parser	Cookie parsing

■ Main Concepts Learned

Through this project, we learn:

1. Node.js and Express setup
2. npm and package management
3. Creating RESTful routes
4. Connecting MongoDB with Mongoose
5. Creating Mongoose models and schemas
6. CRUD operations
7. EJS templates
8. EJS layouts using EJS-Mate
9. PUT and DELETE requests using Method-Override
10. Server-side validation using Joi
11. Error handling with custom `ExpressError`
12. Middleware in Express
13. Sessions using Express-Session
14. Flash messages using Connect-Flash
15. Cookies using Cookie-Parser
16. RESTful backend architecture

■ Install All Direct Packages

Install all direct project dependencies with:

```
npm i express mongoose ejs ejs-mate method-override joi express-session connect-flash cookie-parser
```

Then check them:

```
npm list --depth=0
```

■ Run the Project

Start the application:

```
node app.js
```

If you have nodemon available:

```
npx nodemon app.js
```

The application can normally be accessed at:

```
http://localhost:8080
```

■ Useful npm Commands

Initialize a project:

```
npm init -y
```

Install a package:

```
npm i package-name
```

Install a package as a development dependency:

```
npm i package-name -D
```

Remove a package:

```
npm uninstall package-name
```

Show direct dependencies:

```
npm list --depth=0
```

Show the complete dependency tree:

```
npm list
```

Check outdated packages:

```
npm outdated
```

■ Important Note About `node\_modules`

The `node\_modules` folder can contain many more packages than the number shown in `package.json`.

For example:

```
package.json
  ↓
Direct dependencies
  ↓
Their dependencies
  ↓
Dependencies of those dependencies
  ↓
node_modules/
```

Therefore, do not manually add every folder from `node\_modules` to `package.json`.

Your `package.json` should normally contain the packages your application directly depends on.

■■■ Project

WanderLust Backend

A learning project built with Node.js, Express.js, MongoDB, Mongoose, EJS, RESTful routing, middleware, validation, sessions, cookies, and CRUD operations.